

Project Report: Chat-Bot

Date: December 17, 2025

Author: Project Team (local repo: amankr0804/chat-bot)

Table of Contents

1. Introduction to Project
2. Tools & Technology Used
3. Snapshots
4. Results and Discussions
5. Conclusions and Future Scope
6. References
7. Data Sheet (If any)
8. Appendices (If any)

List of Tables

- Table 1 — Feature checklist and status (see chapter 4)

List of Figures

- Figure 1 — Main chat interface screenshot (assets/screenshots/chat_main.png)
- Figure 2 — Login / Signup screenshot (assets/screenshots/auth.png)

Abbreviations and Nomenclature

- HTML: HyperText Markup Language
- CSS: Cascading Style Sheets
- JS: JavaScript
- UI: User Interface
- UX: User Experience
- API: Application Programming Interface
- DB: Database
- RTDB: Realtime Database

Chapter 1 — Introduction to Project

This project implements a browser-based chat application that provides a simple conversational interface (a rule-based chatbot) and a light authentication flow. The app is built with plain HTML, CSS and JavaScript. The primary goals were:

- Provide a friendly chat UI with persistent chat history

- Create a login/signup flow (localStorage-based fallback)
- Implement attractive responsive styling and animations
- Prepare the app for future cloud synchronization

Project repository: <https://github.com/amankr0804/chat-bot>

Chapter 2 — Tools & Technology Used

- Frontend: `index.html`, `auth.html`, `style.css`, `script.js`
 - HTML5 for structure
 - CSS3 for styling, animations, and responsive layout
 - Vanilla JavaScript for app logic and persistence
- Local persistence: `localStorage` (chat history, users, theme)
- Version control: `git` and GitHub (remote: `amankr0804/chat-bot`)
- Editor / Environment: Visual Studio Code (or any text editor) on Windows
- Fonts: Google Fonts (`Poppins`)
- Optional (attempted): Firebase (Realtime Database & Auth) — attempted but reverted to `localStorage` fallback due to SDK loading issues; documentation for integration available in project notes.

Chapter 3 — Snapshots

Place screenshots in `assets/screenshots/` and name them as referenced below. To include them in the report, add the files and use Markdown image links.

- Figure 1 — Main chat interface: `assets/screenshots/chat_main.png`
- Figure 2 — Authentication page: `assets/screenshots/auth.png`

Example Markdown to include a screenshot:

 Chat main

Chapter 4 — Results and Discussions

Feature summary (Table 1):

- Authentication: Sign-up and login implemented with a client-side `localStorage` approach. The `auth.html` page performs validation and stores users in `localStorage` for demo/testing.
- Chat UI: Fully responsive, modern styling, animated SVG avatars, typing indicator, dark mode, and clear-history button.
- Persistence: Messages are saved to `localStorage` and restored on page load.
- Cloud sync: Firebase integration was attempted and documented but encountered SDK initialization issues; a compat approach was tried and later rolled back in favor of a functional `localStorage`-only flow.

Discussion:

- Stability: The app is stable for local/demo usage with the `localStorage` backend. It supports multiple users on a single machine (separate accounts stored

locally).

- Limitations: No server-side persistence or multi-device sync in the current state. Firebase attempts failed due to `firebase is not defined` / SDK loading ordering while developing directly in the `file://` environment.
- UX: The interface emphasizes clarity and responsiveness. The bot replies are rule-based and deterministic with randomized selections for variety.

Chapter 5 — Conclusions and Future Scope

Conclusions:

- The project meets the primary goals for a local, browser-based chat demo with login/signup and persistent chat history.
- The UI and UX improvements make the app approachable for demonstration and further development.

Future scope / enhancements:

1. Resolve Firebase SDK integration or migrate to a simple backend (Node/Express) with persistent DB (e.g., Firestore, PostgreSQL).
2. Move authentication off `localStorage` to a secure server-side auth (OAuth, JWT, or Firebase Auth when integrated).
3. Add natural language processing (NLP) via a simple intent matcher or integration with an LLM/service for richer replies.
4. Add tests and CI (GitHub Actions) to run linting and basic checks on commits.
5. Add end-to-end monitoring and analytics for usage metrics.

References

- Project repository: <https://github.com/amankr0804/chat-bot>
- Firebase documentation: <https://firebase.google.com/docs>
- Google Fonts — Poppins: <https://fonts.google.com/specimen/Poppins>

Data Sheet

No external sensor data sheets are required for this project. If you have a specific data sheet (input/output format, API schema), include it in this section.

Appendices

Key source files (in this repository):

- Main chat page: [index.html](#)
- Authentication page: [auth.html](#)
- Styles: [style.css](#)
- Client logic: [script.js](#)

Notes:

- To run locally, open `auth.html` in the browser, create an account, then you will be redirected to `index.html` for chat.
 - To add screenshots for chapter 3, create `assets/screenshots/` and add the PNG(s). Then commit and push.
-

If you'd like, I can:

- Commit `REPORT.md` to the repository and push it to GitHub.
- Generate a PDF export of this markdown.
- Insert actual screenshots if you upload them to `assets/screenshots/`.

Generated from REPORT.md